Microsoft Purview Tenant Assessment

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Tenant Assessment 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

Approach . . . 5

Boundaries . . . 5

The bottom line . . . 6

1. Solution Overview — Microsoft Purview Tenant Assessment 6

What this solution is . . . 6

Why this matters . . . 6

How it works . . . 7

The capabilities this solution assesses . . . 7

What you get out of it . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 9

The standard deployment model . . . 9

2. Requirements Guide — Microsoft Purview Tenant Assessment 9

2.1 Compliance licensing across the capabilities in scope . . . 10

2.2 Least-privilege read roles across every capability (Reader-first) . . . 10

2.3 Security & Compliance / Exchange Online PowerShell + Graph . . . 10

2.4 Confirmed capability inventory (what's actually enabled) . . . 10

2.5 Write roles only for the quick-win remediations (if remediating) . . . 11

2.6 Stakeholders for the findings that need a governance decision . . . 11

2.7 A remediation pilot scope and change window (if remediating) . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Microsoft Purview Tenant Assessment 14

Step 1 — Preflight the scope, licensing, and roles (Assess) . . . 14

Step 2 — Inventory every enabled capability (Assess) . . . 14

Step 3 — Design the findings and priority scheme (Design) . . . 15

Step 4 — Produce the findings register, priorities, and scorecard (Design) . . . 15
Step 5 — Configure the safe quick-win fixes — simulation first (Configure → Pilot) . . . 16
Step 6 — Validate the remediated findings clear (Validate) . . . 16
Step 7 — Readout, expand, and track (Expand) . . . 16
5. Testing & Validation — Microsoft Purview Tenant Assessment 17
Gate 1 — Preflight is clean (licensing + roles + endpoints) . . . 17
Gate 2 — Every enabled capability is inventoried . . . 17
Gate 3 — DLP coverage and mode are captured correctly . . . 17
Gate 4 — Labels, retention, and audit states are read correctly . . . 18
Gate 5 — Portal-only capabilities are flagged, not fabricated . . . 18
Gate 6 — Findings register is complete and prioritized . . . 18
Gate 7 — Scorecard reflects the inventory . . . 18
Gate 8 — Quick-win remediations previewed before commit . . . 18
Gate 9 — Remediated findings validate green . . . 19
Gate 10 — Readout delivered and backlog owned . . . 19
Evidence to retain . . . 19
6. Troubleshooting — Microsoft Purview Tenant Assessment 19
Connectivity & authentication . . . 19
Roles & permissions (a permission gap looks like a coverage gap) . . . 20
Discovery / inventory . . . 20
Remediation (simulation-first) . . . 20
Validation . . . 21
Scope of these gates . . . 21
7. Best Practices — Assessing a Microsoft Purview Tenant 21
Assess the whole estate, not one capability at a time . . . 21
Read-only first; never let a permission gap look like a coverage gap . . . 21
Be honest about what PowerShell can and can't read . . . 21
Prioritize by risk, and make the priorities mean something . . . 22
Every fix is simulation/test/pilot-first — even the "safe" ones . . . 22
Keep the governance line bright . . . 22
Make it a baseline you can re-run . . . 22
Least-privilege, evidenced, reversible . . . 22
8. Deliverables Summary — Microsoft Purview Tenant Assessment 23
Documentation . . . 23
PowerShell tools (5) . . . 23
Named artifacts produced . . . 24

Definition of done 24
Appendix A — PowerShell Scripts 26

Executive Summary — Microsoft Purview Tenant Assessment

The problem

Microsoft Purview capabilities get turned on one at a time — a DLP policy, a few sensitivity labels, retention when legal asks, Insider Risk after an incident — and no one ever steps back to ask the whole-estate question: *across every capability we've enabled, what is actually configured, where are the gaps, and what should we fix first?* The result is a compliance blind spot: DLP that covers email but not Teams, a label taxonomy published to no one, a retention policy sitting **disabled** for a year, an Insider Risk policy running with no reviewers — none of it visible, and therefore none of it fixed.

The solution

A complete, self-guided **tenant-wide Purview assessment**. It inventories **every enabled Purview capability** — Information Protection, DLP, Data Lifecycle & Records, Insider Risk, Communication Compliance, Audit, eDiscovery, DSPM (and DSPM for AI), Compliance Manager, the Unified Catalog, and the classifiers & sensitive information types underneath — reads each read-only, scores it on coverage and maturity, and produces a **prioritized findings register (P1/P2/P3)** with a recommendation per finding. Where a quick, safe win exists, it remediates **in simulation/test/pilot mode first**, validates, and hands over a defensible baseline. Five safe-by-default PowerShell tools (read-only, or `-WhatIf` until you commit) do the preflight, inventory, remediation, validation, and reporting.

What you get

- **The full picture** — an evidenced inventory of every enabled capability's current configuration, with **not licensed / not enabled / portal-only** stated honestly rather than guessed.
- **A prioritized plan** — findings ranked P1 (critical exposure / enforced-but-risky), P2 (coverage gap / drift), P3 (optimization), so remediation follows risk.
- **A capability scorecard** — coverage and maturity at a glance, capability by capability.
- **Safe stabilization** — the quick wins (a blunt DLP policy moved to simulation, a disabled retention policy enabled, unified audit turned on) applied pilot-first with rollback.
- **An executive readout** and a governed backlog the compliance owners can act on and track quarter over quarter.

Approach

Delivery follows a disciplined, reversible model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — reading the whole estate first, prioritizing by risk, and applying any fix **simulation/test/pilot-first** on a limited scope, with pass/fail gates and documented rollback at every step.

Boundaries

Microsoft Purview capabilities are **licensing-/plan-dependent** (many require E5 or the E5 Compliance add-on; some are preview) and several are **portal-driven** with limited PowerShell — availability and paths are cited or flagged **"verify your licensing/tenant"** and **"portal-only — review manually,"** never

invented. **Whether a policy is correct — retention durations, a rule's regulatory sufficiency, whether a capability is required, any records or legal-hold obligation — is deferred to your compliance, records, privacy, and legal functions.** The assessment provides the technical findings and evidence, not the determination; Insider Risk and Communication Compliance are reviewed for configuration hygiene and role separation only.

The bottom line

The whole Purview estate baselined in one pass — every enabled capability inventoried, the gaps found and prioritized, the quick wins safely fixed — so you finally have **one picture, a ranked plan, and a defensible baseline** instead of a blind spot.

1. Solution Overview — Microsoft Purview Tenant Assessment

What this solution is

Most organizations turn on Microsoft Purview capabilities one at a time — a DLP policy here, a few sensitivity labels there, retention when legal asks, Insider Risk after an incident — and nobody ever steps back to answer the whole-estate question: *across every Purview capability we have enabled, what is actually configured, where are the gaps, and what should we fix first?* This solution delivers that **tenant-wide Purview assessment**. It inventories **every enabled Purview capability** — Information Protection (sensitivity labels), Data Loss Prevention, Data Lifecycle & Records Management (retention), Insider Risk Management, Communication Compliance, Audit, eDiscovery, Data Security Posture Management (DSPM & DSPM for AI), Compliance Manager, the Unified Catalog / Data Governance data map, and the classifiers & sensitive information types underneath them — reads their current configuration read-only, scores each on coverage and maturity, produces a **prioritized findings register (P1/P2/P3)** with recommendations, and — where a quick, safe win exists — remediates it **in simulation/test mode first**, validates, then hands you a defensible baseline. It uses only the **Microsoft Purview portal**, **Security & Compliance / Exchange Online PowerShell**, and **Microsoft Graph** — no third-party tooling.

Why this matters

- **Nobody has the full picture.** Config accumulates across teams and years; without a single assessment you can't see that DLP covers email but not Teams, that a label taxonomy exists but was never published, or that a retention policy has been sitting **disabled** for a year. Gaps you can't see, you can't fix.
- **Drift is silent.** Policies get created in enforce mode without a pilot, labels get orphaned, an auto-labeling policy stays stuck in simulation, an Insider Risk policy runs with no reviewers. An assessment surfaces the drift as concrete, evidenced findings — not opinions.
- **Priorities beat activity.** A tenant can have fifty findings; what matters is *which three* to fix this quarter. This solution ranks findings **P1 (critical exposure / enforced-but-risky), P2 (coverage gap / drift), P3 (optimization)** so remediation follows risk, not whoever shouts loudest.

• **The controls are licensing-dependent and the portal evolves.** Many Purview capabilities require **Microsoft 365 E5** or the **E5 Compliance** add-on, and some (DSPM for AI, parts of the Unified Catalog) are in preview or actively changing. The assessment reports **what your tenant actually has enabled and licensed** — it never assumes a capability is present.

How it works

1. **Preflight the scope.** Confirm licensing, least-privilege roles, and which Purview capabilities are actually enabled/visible in *your* tenant — so the assessment covers what exists, not a generic checklist.

2. **Inventory every enabled capability (read-only).** For each capability, read its configuration: labels & label policies, DLP policies and their mode, retention policies/labels and whether they're enabled, Insider Risk & Communication Compliance policy presence and reviewer/scope hygiene, Audit ingestion state, eDiscovery cases/holds, DSPM coverage, the Compliance Manager score, catalog/data-map state, and the SITs/classifiers in use.

3. **Score and find.** Each capability gets a coverage/maturity read; each concrete gap becomes a **finding** with capability, observation, evidence source, risk, recommendation, and a **priority**. Regulatory/records/legal judgments are recorded as findings **for the relevant governance function to decide** — never decided here.

4. **Prioritize.** Findings roll up into `Purview Assessment – Priorities – P1P2P3` and a per-capability `Purview Assessment – Capabilities – Scorecard` so the readout is a plan, not a data dump.

5. **Remediate the safe wins — simulation first.** Where a finding has a low-risk, high-value fix (e.g. move a bluntly-enforced DLP policy into **simulation/test**, take an auto-labeling policy to **simulation** before enforce, enable a retention policy left disabled, turn on unified audit), the remediation tool applies it **pilot-scoped and in simulation/test/**`-WhatIf` **first**, never straight to tenant-wide enforce.

6. **Validate and hand over.** Re-check that remediated findings pass their gate, produce the `Purview Assessment – Tenant – Readout`, and hand the register + scorecard to the compliance owners as the standing baseline to track.

The capabilities this solution assesses

#	Capability	What the assessment reads
1	**Information Protection**	Sensitivity labels, label policies (published scope), auto-labeling policies + their mode
2	**Data Loss Prevention (DLP)**	DLP policies + rules, per-workload coverage (Exchange, SharePoint, OneDrive, Teams, Endpoint), policy **mode** (enforce vs test)
3	**Data Lifecycle & Records Management**	Retention policies & retention labels, **enabled** state, label-policy publication, disposition

#	Capability	What the assessment reads
4	**Insider Risk Management**	Policy presence, template/scope, reviewer assignment hygiene (privacy-by-design)
5	**Communication Compliance**	Policy presence, reviewer groups, scope hygiene
6	**Audit (Standard/Premium)**	Unified audit ingestion state, retention policies, licensing tier
7	**eDiscovery (Standard/Premium)**	Compliance cases, hold policies, case hygiene
8	**Data Security Posture Management (DSPM & DSPM for AI)**	Coverage/enablement (portal-driven; flagged where preview)
9	**Compliance Manager**	Overall compliance score and improvement-action posture
10	**Unified Catalog / Data Governance**	Data map/catalog enablement and coverage (flagged where evolving)
11	**Classifiers & Sensitive Info Types**	Custom SITs, trainable classifiers, EDM schemas in use across the above

> ***Not every capability is enabled or licensed in every tenant.*** *The assessment reports each as* ***enabled / not enabled / not licensed / portal-only — review manually****, and flags anything it cannot read reliably as* ***"verify your licensing/tenant"*** *rather than asserting a value.*

What you get out of it

- A **complete, evidenced inventory** of every enabled Purview capability's current configuration.
- A **prioritized findings register** (P1/P2/P3) with a recommendation per finding.
- A **per-capability scorecard** showing coverage/maturity at a glance.
- **Safe remediation of the quick wins** — applied in simulation/test/pilot first, with rollback.
- An **executive readout** the compliance owners can act on and track quarter over quarter.

Limitations (be honest with buyers)

- **This is an assessment, not a full build-out.** It baselines what exists and safely fixes quick wins; standing up a capability from scratch (e.g. a full DLP program or a records file plan) is a separate implementation engagement. Findings point to those.
- **Capabilities are license-gated and some are preview.** Insider Risk, Communication Compliance, Audit Premium, DSPM for AI, auto-labeling, and parts of the Unified Catalog require **E5 / E5 Compliance** (or are in preview). Where a capability isn't licensed/enabled, the assessment records that

fact — it doesn't fabricate a reading. **Verify your licensing/tenant.**

• **Some capabilities are portal-driven with limited/no stable PowerShell surface** (notably Insider Risk configuration, DSPM, Compliance Manager scoring, and the evolving Unified Catalog). For those, the tools read what is reliably readable and otherwise flag **"portal-only — review manually"** with the exact portal location — they never invent a cmdlet.

• **Regulatory, records, and legal determinations are not ours to make.** Whether a retention duration is *correct*, whether a DLP rule *satisfies* a regulation, whether an Insider Risk case *warrants* action — these are **deferred to your compliance, records, privacy, and legal functions**. The assessment provides the technical finding and evidence, not the determination.

• **Insider Risk & Communication Compliance are HR/privacy-sensitive.** The assessment reviews them for **configuration hygiene and role separation only** (privacy-by-design), never case content.

Naming convention (used in every doc and script)

Every artifact this assessment produces is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and portal. The standard set:

Artifact	Name
Prioritized findings register	`Purview Assessment – Tenant – Findings`
Priority matrix (P1/P2/P3)	`Purview Assessment – Priorities – P1P2P3`
Per-capability coverage/maturity scorecard	`Purview Assessment – Capabilities – Scorecard`
Remediation pilot scope	`Purview Assessment – Remediation Pilot – Stage1`
Executive readout	`Purview Assessment – Tenant – Readout`

The standard deployment model

> *Assess what every enabled capability actually is → design the prioritized findings and target state → configure the safe quick-win fixes → **pilot them in simulation/test/**`-WhatIf` on a limited scope first → validate the findings clear → expand/track the rest as a governed backlog. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand.***

2. Requirements Guide — Microsoft Purview Tenant Assessment

Verify **every** requirement before starting the assessment. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant/licensing, Purview roles, the S&C PowerShell + Graph toolchain, in-scope workloads, compliance/records/legal stakeholders, pilot & change control) are in `global-requirements-purview.md` — this guide adds the assessment-specific detail.

> ***Licensing/tenant boundary.** Which Purview capabilities you can assess depends on what your tenant has **enabled and licensed**. Advanced capabilities (auto-labeling, advanced DLP, Insider Risk,*

*Communication Compliance, Audit Premium, DSPM/DSPM for AI, parts of the Unified Catalog) require **Microsoft 365 E5** or the **E5 Compliance** add-on, and some are in **preview**. Where this guide flags "verify your licensing/tenant," confirm the entitlement before relying on a reading. A capability that is **not licensed / not enabled** is a legitimate assessment result, not a defect.*

2.1 Compliance licensing across the capabilities in scope

Description: The Microsoft 365 compliance licensing for the capabilities you want assessed — **E3** covers baseline labels/DLP/retention/Audit Standard; **E5 / E5 Compliance** unlocks auto-labeling, advanced DLP, Insider Risk, Communication Compliance, Audit Premium, DSPM, and eDiscovery Premium. **Why:** The assessment can only read a capability that is licensed and enabled; unlicensed capabilities are reported as such, not scored (GR §1). **Verify:** M365 admin center → `Billing → Licenses`; and run `PURVASSESS-Scope.ps1`, which reads `subscribedSkus`/service plans via Graph and reports which Purview-relevant plans are present. **Implement:** No purchase is required to *assess* — the assessment simply records what's licensed. Findings will recommend where a missing license blocks a needed capability; that decision is the customer's.

2.2 Least-privilege read roles across every capability (Reader-first)

Description: Read access to each capability being assessed. The **Global Reader** and **Compliance Administrator** / **Compliance Data Administrator** roles cover most reads; capability-specific reader roles may be needed for **Insider Risk** (Insider Risk Management **Analysts/Investigators** or a read role), **eDiscovery** (eDiscovery **Reader/Manager**), and **Audit** (**Audit Reader**). Reserve Global Administrator for the rare enablement action only. **Why:** Purview uses fine-grained role groups; a read the account isn't entitled to simply returns nothing or an access error — which would look like a false "gap" (GR §2). **Verify:** Purview portal → `Settings → Roles & scopes / Permissions`; `PURVASSESS-Scope.ps1` reports which capability reads succeed and which throw a permissions error. **Implement:** Assign the least-privilege **reader** role groups for the in-scope capabilities under change control. Prefer read roles for the assessment; the remediation step (2.5) needs write roles only for the specific quick-win capabilities.

2.3 Security & Compliance / Exchange Online PowerShell + Graph

Description: The `ExchangeOnlineManagement` module provides **Security & Compliance PowerShell** (`Connect-IPPSSession` — labels, DLP, retention, some Communication Compliance/eDiscovery cmdlets) and **Exchange Online** (`Connect-ExchangeOnline` — audit config, mailbox settings). **Microsoft.Graph** is used for licensing, users/roles, and (where available) security/compliance reads. **Why:** The assessment tools read across all three; the right endpoint is used for each capability (GR §3). **Note:** several capabilities (Insider Risk, DSPM, Compliance Manager scoring, the Unified Catalog) are **portal-driven** and have limited or no stable PowerShell — the tools flag those for manual portal review rather than invent cmdlets. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement`; both `Connect-IPPSSession` and `Connect-ExchangeOnline` succeed with a modern-auth/MFA admin; `Connect-MgGraph` succeeds. `PURVASSESS-Scope.ps1` reports which endpoints are reachable. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.4 Confirmed capability inventory (what's actually enabled)

Description: A confirmed list of **which Purview capabilities your tenant has enabled** — not the theoretical catalog. Some may be licensed but never turned on (e.g. Insider Risk enabled but no policies, DSPM not yet activated). **Why:** The assessment scopes to enabled capabilities; "licensed but never configured" is itself a headline finding, and the tool must distinguish *not licensed* from *licensed-but-empty* (GR §4). **Verify:** In the Purview portal, note which solutions appear under `Solutions`; `PURVASSESS-Scope.ps1` + `PURVASSESS-Discovery.ps1` report each capability as enabled / not enabled / not licensed / portal-only. **Implement:** No action needed to assess — the inventory is an *output*. Just ensure the roles (2.2) let the tools read each enabled capability.

2.5 Write roles only for the quick-win remediations (if remediating)

Description: If you intend to apply the safe quick-win fixes (put a DLP policy into simulation, take an auto-labeling policy to simulation, enable a disabled retention policy, enable unified audit), the account needs the **write** role for *those specific capabilities* (e.g. **Compliance Administrator** for DLP/retention/labels; **Audit Manager** / Exchange for audit ingestion) — assigned under change control. **Why:** Remediation changes configuration; it must be least-privilege and change-controlled, and it is always **simulation/test/pilot-first** (GR §6). **Verify:** `PURVASSESS-Remediate.ps1 -WhatIf` prints the exact cmdlet it *would* run and reports a permissions error if the write role is missing — nothing changes until `-Commit`. **Implement:** Grant the specific write role for the capability being remediated, under an approved change window; remove it after if it was granted just for the fix.

2.6 Stakeholders for the findings that need a governance decision

Description: Named owners in **compliance, records, privacy, security, and legal** who will receive the findings that are **determinations, not technical fixes** — is a retention duration correct, does a DLP rule satisfy a regulation, should an Insider Risk policy exist, is a capability required. **Why:** Regulatory/records/legal judgments are theirs to make (GR §5); the assessment produces the finding and evidence, the governance function decides. **Verify:** You can name the owner for each capability's policy decisions. **Implement:** Identify the owners before the readout so P1/P2 findings that need a decision have a destination.

2.7 A remediation pilot scope and change window (if remediating)

Description: A limited **pilot scope** (`Purview Assessment – Remediation Pilot – Stage1`) — a small set of users/policies to apply and validate any quick-win fix on before tenant-wide, plus an agreed change window (GR §6). Every Purview policy change here uses **simulation/test/disabled** mode first. **Why:** Even "safe" fixes (moving a DLP policy's mode, enabling a retention policy) are tenant-impacting; validate small first. **Verify:** A named pilot scope, confirmation that simulation/test mode is available for the target policy, and change approval exist. **Implement:** Define the pilot and change window before running `PURVASSESS-Remediate.ps1 -Commit` on anything.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

Vendor boundary (applies to every Purview solution). *The* ***Microsoft Purview portal*** *(purview.microsoft.com), the* ***Microsoft 365 admin center****,* ***Security & Compliance PowerShell*** *(`Connect-IPPSSession`), and* ***Microsoft Graph*** *are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily* ***licensing-/plan-dependent*** *(many require* ***Microsoft 365 E5*** *or the* ***E5 Compliance*** *add-on; some are in preview) and the portal is* ***actively evolving*** *— capability availability and portal paths are* ***cited or flagged "verify your licensing/tenant"****, never asserted or invented.* ***Regulatory, records-retention, and legal determinations*** *(retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are* ***deferred to the organization's compliance, records, privacy, and legal functions*** *— the solutions provide the technical configuration method and evidence, not the determination. Automation is* ***PowerShell*** *(Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Tenant Assessment

Follow these steps **in order**. The assessment uses the **Microsoft Purview portal** (`https://purview.microsoft.com`), **Security & Compliance PowerShell** (`Connect-IPPSSession` — labels, DLP, retention, and where available Communication Compliance/eDiscovery), **Exchange Online PowerShell** (`Connect-ExchangeOnline` — audit config), and **Microsoft Graph** (licensing, roles, security reads). Every discovery/validation/report tool is **read-only**; the remediation tool runs `-WhatIf` **by default** and changes state only with `-Commit`, always **simulation/test/pilot-first**. Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:*** *Assess every enabled capability, design the prioritized findings, configure the safe quick-win fixes,* ***pilot*** *them in simulation/test on a limited scope, validate the findings clear, then expand/track the rest as a governed backlog. Pattern:* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot (simulate)*** *→* ***Validate*** *→* ***Expand.***

Step 1 — Preflight the scope, licensing, and roles (Assess)

1. Run PURVASSESS-`Scope.ps1` (read-only). It connects to S&C PowerShell, Exchange Online, and Graph and reports:

• which Purview-relevant **service plans** are present (E5 / E5 Compliance signals) — *licensed vs not*,

• which capability **reads succeed** vs throw a permissions error (role coverage — Requirement 2.2),

• which endpoints (`Connect-IPPSSession`, `Connect-ExchangeOnline`, `Connect-MgGraph`) are reachable.

1. In the Purview portal, note which capabilities appear under `Solutions` (visibility depends on role + licensing).

2. **Pass gate:** you have a written statement of what is licensed and readable, and any role/endpoint gap is resolved before assessing — so a "gap" finding is a real gap, not a missing permission.

Step 2 — Inventory every enabled capability (Assess)

1. Run PURVASSESS-`Discovery.ps1` (read-only). For each capability it reads and records, tagging each as **enabled / not enabled / not licensed / portal-only — review manually**:

• **Information Protection** — `Get-Label`, `Get-LabelPolicy`, `Get-AutoSensitivityLabelPolicy` (labels, publication scope, auto-labeling mode).

• **DLP** — `Get-DlpCompliancePolicy`, `Get-DlpComplianceRule` (policies, per-workload locations, **mode** enforce vs test).

• **Data Lifecycle & Records** — `Get-RetentionCompliancePolicy`, `Get-RetentionComplianceRule`, `Get-ComplianceTag` (policies/labels, **enabled** state, publication).

• **Audit** — `Get-AdminAuditLogConfig` (ingestion), `Get-UnifiedAuditLogRetentionPolicy` (retention).

• **eDiscovery** — `Get-ComplianceCase`, `Get-CaseHoldPolicy` (cases, holds).

• **Communication Compliance** — read where the tenant exposes it (flagged portal-only if the read isn't available).

• **Insider Risk**, **DSPM / DSPM for AI**, **Compliance Manager**, **Unified Catalog** — **portal-driven**; the tool records them as *review manually* with the exact portal location (it does not invent cmdlets).

• **Classifiers & SITs** — `Get-DlpSensitiveInformationType`, trainable-classifier reads where available.

1. The tool writes the raw inventory that Step 4 turns into findings. **Pass gate:** every enabled capability has a recorded current-state, and every not-enabled/not-licensed/portal-only one is explicitly marked (not silently skipped).

Step 3 — Design the findings and priority scheme (Design)

1. Agree the **priority definitions** with your stakeholders (Requirement 2.6), used verbatim in the register:

• **P1** — critical exposure or enforced-but-risky: sensitive data unprotected, a DLP/label/retention policy enforced tenant-wide with no pilot, unified audit off, an Insider Risk/Communication Compliance policy running with no reviewers.

• **P2** — coverage gap or drift: a workload not covered (e.g. DLP on email but not Teams), a label taxonomy published to no one, an auto-labeling policy stuck in simulation, a retention policy left **disabled**, a licensed-but-empty capability.

• **P3** — optimization: redundant/overlapping policies, naming inconsistency, unused SITs, tuning opportunities.

1. Agree that **regulatory/records/legal** findings (is a duration correct, does a rule satisfy a regulation) are recorded as **decisions for the governance owner**, not fixed here (Requirement 2.6).

2. Define the **remediation pilot scope** `Purview Assessment – Remediation Pilot – Stage1` (Requirement 2.7). **Pass gate:** priority definitions, decision routing, and pilot scope are documented and approved.

Step 4 — Produce the findings register, priorities, and scorecard (Design)

1. Run `PURVASSESS-Report.ps1` (read-only). From the Step 2 inventory it produces:

• `Purview Assessment – Tenant – Findings` — every finding with **capability · observation · evidence source · risk · recommendation · priority**.

• `Purview Assessment – Priorities – P1P2P3` — the findings rolled up by priority.

• `Purview Assessment – Capabilities – Scorecard` — each capability's coverage/maturity read (and enabled/not-enabled/portal-only state).

1. Review the register with stakeholders; route the determination findings to their owners (Requirement 2.6). **Pass gate:** the register is reviewed and each P1/P2 has an owner and a decision (fix here / decide by governance / defer to a build engagement).

Step 5 — Configure the safe quick-win fixes — simulation first (Configure → Pilot)

Only the low-risk, high-value fixes are applied here; standing up a whole capability is a separate engagement (the finding points to it). Every fix is **pilot-scoped and simulation/test/**`-WhatIf` **first**. Each of these prints the exact cmdlet under `-WhatIf`, then applies on `-Commit`:

1. **Move a bluntly-enforced DLP policy into simulation** (so you can see impact before it blocks users):

`PURVASSESS-Remediate.ps1 -Fix DlpToSimulation -PolicyName "<policy>" -WhatIf` → review → `-Commit`. (Runs `Set-DlpCompliancePolicy -Mode TestWithNotifications`.)

1. **Take an auto-labeling policy to simulation** before enforcement:

`PURVASSESS-Remediate.ps1 -Fix AutoLabelToSimulation -PolicyName "<policy>" -WhatIf` → review → `-Commit`.

1. **Enable a retention policy that was left disabled** (after the records owner confirms it should run — Requirement 2.6):

`PURVASSESS-Remediate.ps1 -Fix EnableRetentionPolicy -PolicyName "<policy>" -WhatIf` → review → `-Commit`. (Runs `Set-RetentionCompliancePolicy -Enabled $true`.)

1. **Enable unified audit ingestion** if it is off:

`PURVASSESS-Remediate.ps1 -Fix EnableUnifiedAudit -WhatIf` → review → `-Commit`. (Runs `Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`.)

1. Scope every fix to `Purview Assessment – Remediation Pilot – Stage1` where the policy supports scoping, and record the `-WhatIf` transcript. **Pass gate:** each applied fix ran `-WhatIf` clean, was reviewed, and committed only after approval. **Rollback:** every fix is reversible — `-Fix DlpToSimulation` back to its prior mode, `-Fix EnableRetentionPolicy` → `Set-RetentionCompliancePolicy -Enabled $false`, audit enablement is non-destructive; `PURVASSESS-Remediate.ps1 -Fix <name> -Revert -Commit` restores the recorded prior state.

Step 6 — Validate the remediated findings clear (Validate)

1. Run `PURVASSESS-Validate.ps1` (read-only). For each fix applied in Step 5 it re-reads the capability and confirms the target state (DLP policy now in `TestWithNotifications`, auto-labeling in simulation, retention policy `Enabled = True`, audit ingestion `True`).

2. Re-run `PURVASSESS-Report.ps1` and confirm the corresponding findings now show **cleared** in `Purview Assessment – Tenant – Findings`. Complete the **Testing Guide** before expanding.

3. **Pass gate:** every remediated finding validates green for the pilot scope.

Step 7 — Readout, expand, and track (Expand)

1. Produce `Purview Assessment – Tenant – Readout` (executive summary of the scorecard + P1/P2/P3 with owners and status) and walk it with stakeholders.

2. For findings **not** fixed here — governance decisions and full-build items — record them as a **governed backlog** with owners and target dates; they are inputs to implementation engagements, not this assessment.

3. Schedule the read-only chain (`PURVASSESS-Scope.ps1` → `PURVASSESS-Discovery.ps1` → `PURVASSESS-Report.ps1`) on a cadence (e.g. quarterly) so the tenant is **re-baselined** and drift is caught — a falling P1/P2 count over successive runs is the measure of progress. **Pass gate:** the readout is delivered, the backlog is owned, and the re-baseline cadence is scheduled.

5. Testing & Validation — Microsoft Purview Tenant Assessment

Work through every gate. Each has a **method**, an **expected result (pass)**, and a **fail action**. The assessment tools are read-only; the only writes are the Step-5 quick-win fixes, applied simulation/test/pilot-first. Do not declare the assessment "delivered" until the inventory is complete, the register is reviewed, and every remediated finding validates for the pilot scope.

> ***Remember:*** *a capability reported as* ***not licensed / not enabled / portal-only*** *is a legitimate result, not a test failure. These gates confirm the assessment *read what exists correctly* and that any quick-win fix took effect — they do not judge whether a policy is *correct* (that is a compliance/records/legal determination).*

Gate 1 — Preflight is clean (licensing + roles + endpoints)

• **Method:** `PURVASSESS-Scope.ps1`.

• **Pass:** every in-scope capability read **succeeds** (no permissions error), all needed endpoints connect, and the licensed/not-licensed picture is recorded.

• **Fail:** resolve the missing role (Requirement 2.2) or endpoint before assessing — otherwise a permission gap masquerades as a coverage gap.

Gate 2 — Every enabled capability is inventoried

• **Method:** `PURVASSESS-Discovery.ps1`.

• **Pass:** each of the 11 capabilities is recorded as **enabled / not enabled / not licensed / portal-only — review manually** — nothing is silently skipped.

• **Fail:** if a capability you know is enabled shows no read, check the role (Gate 1) and endpoint; if it is genuinely portal-only, confirm it is flagged for manual review with the portal location.

Gate 3 — DLP coverage and mode are captured correctly

• **Method:** `PURVASSESS-Discovery.ps1` DLP section vs the Purview portal DLP policy list.

• **Pass:** every DLP policy is listed with its **workload locations** and **mode** (enforce vs test); an enforced-tenant-wide-without-pilot policy is flagged as a **P1** finding.

• **Fail:** re-read with the correct role; confirm the mode value matches the portal.

Gate 4 — Labels, retention, and audit states are read correctly

• **Method:** `PURVASSESS-Discovery.ps1` sections for Information Protection, Data Lifecycle, and Audit.

• **Pass:** published-label scope, auto-labeling mode, retention-policy **enabled** state, and `UnifiedAuditLogIngestionEnabled` all match the portal; a **disabled** retention policy and **audit off** each surface as findings.

• **Fail:** confirm the endpoint (retention/labels via `Connect-IPPSSession`; audit via `Connect-ExchangeOnline`) and re-read.

Gate 5 — Portal-only capabilities are flagged, not fabricated

• **Method:** `PURVASSESS-Discovery.ps1` output for Insider Risk, Communication Compliance, DSPM, Compliance Manager, Unified Catalog.

• **Pass:** each is marked **portal-only — review manually** with the exact portal location, or read only where a reliable cmdlet exists — never asserted with an invented value.

• **Fail:** if any shows an unverifiable value, treat it as a bug; the honest result is "review manually / verify your licensing/tenant."

Gate 6 — Findings register is complete and prioritized

• **Method:** `PURVASSESS-Report.ps1` → `Purview Assessment – Tenant – Findings` + … `Priorities – P1P2P3`.

• **Pass:** every finding has **capability · observation · evidence source · risk · recommendation · priority**, and the P1/P2/P3 roll-up matches the register.

• **Fail:** fill the missing field; re-run the report.

Gate 7 — Scorecard reflects the inventory

• **Method:** `PURVASSESS-Report.ps1` → `Purview Assessment – Capabilities – Scorecard`.

• **Pass:** each capability's coverage/maturity read and enabled/not-enabled/portal-only state agrees with the Step-2 inventory.

• **Fail:** reconcile the scorecard to the inventory; re-run.

Gate 8 — Quick-win remediations previewed before commit

• **Method:** `PURVASSESS-Remediate.ps1 -Fix <name> ... -WhatIf` for each intended fix.

• **Pass:** `-WhatIf` prints the exact cmdlet it would run and makes **no change**; a permissions error (if any) is reported rather than a silent failure.

• **Fail:** resolve the write role (Requirement 2.5) or the parameter, and re-preview — never commit an unreviewed fix.

Gate 9 — Remediated findings validate green

- **Method:** `PURVASSESS-Validate.ps1`, then re-run `PURVASSESS-Report.ps1`.
- **Pass:** each committed fix shows its target state (DLP in `TestWithNotifications`, auto-labeling in simulation, retention `Enabled = True`, audit ingestion `True`) and the corresponding finding shows **cleared** for the pilot scope.
- **Fail:** re-check the fix and the pilot scope; if it didn't take, revert (`-Revert -Commit`) and investigate before retrying.

Gate 10 — Readout delivered and backlog owned

- **Method:** `Purview Assessment – Tenant – Readout` reviewed with stakeholders.
- **Pass:** the readout presents the scorecard + prioritized findings; every non-remediated P1/P2 has an owner and a route (governance decision / build engagement / deferred), and the re-baseline cadence is scheduled.
- **Fail:** assign the missing owners/dates before closing the engagement.

Evidence to retain

- `PURVASSESS-Scope.ps1` and `PURVASSESS-Discovery.ps1` outputs (the baseline inventory).
- `Purview Assessment – Tenant – Findings`, … `Priorities – P1P2P3`, … `Capabilities – Scorecard`.
- The `-WhatIf` transcripts and `-Commit` confirmations for every remediation (Requirement 2.7).
- `PURVASSESS-Validate.ps1` output showing findings cleared, and the change approvals.

*Whether a policy is **correct** — a retention duration, a DLP rule's regulatory sufficiency, whether a capability is *required* — is a **compliance/records/legal determination**. These gates prove the assessment read the tenant correctly and the fixes took effect, not that the resulting configuration satisfies a regulation.*

6. Troubleshooting — Microsoft Purview Tenant Assessment

Common issues, grouped by stage. Each has a **symptom**, the **likely cause**, and a **resolution**. The assessment tools are read-only except `PURVASSESS-Remediate.ps1`, which is `-WhatIf` by default and writes only with `-Commit` — so most "it didn't work" issues here are connection, role, or licensing gaps rather than damage. When a capability legitimately reads as **not licensed / not enabled / portal-only**, that is a *finding*, not a failure to fix.

Connectivity & authentication

Symptom: `Connect-IPPSSession` or `Connect-ExchangeOnline` fails or hangs. **Likely cause:** the `ExchangeOnlineManagement` module is missing/outdated, MFA prompt was missed, or a conditional-access policy blocks the sign-in. **Resolution:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` (or `Update-Module`), complete the interactive

modern-auth prompt, and pass `-UserPrincipalName` so the right admin is used. Confirm the account is allowed by conditional access for PowerShell.

Symptom: the Graph licensing section is skipped or errors. **Likely cause:** `Microsoft.Graph` module not installed, or consent not granted for `Organization.Read.All` / `Directory.Read.All`. **Resolution:** `Install-Module Microsoft.Graph -Scope CurrentUser`, re-run and consent, or run `PURVASSESS-Scope.ps1 -SkipGraph` and confirm licensing manually in the Microsoft 365 admin center.

Roles & permissions (a permission gap looks like a coverage gap)

Symptom: `PURVASSESS-Scope.ps1` reports a capability read as **FAILED (role/permission)**. **Likely cause:** the signed-in admin lacks the least-privilege role for that capability (Requirement 2.2). **Resolution:** grant the reader role (e.g. **Compliance Administrator** / **Global Reader** / capability- specific reader) in the Purview portal `Roles & scopes` or Entra, wait for propagation (can take minutes), and re-run. **Do not** record the capability as a gap until the read succeeds — otherwise you flag a missing permission as a missing control.

Symptom: `PURVASSESS-Remediate.ps1 -Commit` throws an access-denied error. **Likely cause:** the write role for that fix is missing (Requirement 2.5) — read access is not enough to change a policy. **Resolution:** grant the appropriate write role, re-preview with `-WhatIf`, then re-run with `-Commit`.

Discovery / inventory

Symptom: a capability you know is enabled shows no data in `PURVASSESS-Discovery.ps1`. **Likely cause:** wrong endpoint for that capability (retention/labels via `Connect-IPPSSession`; audit via `Connect-ExchangeOnline`), or a role gap (see above). **Resolution:** confirm the endpoint connected in Step 1, resolve any Gate-1 read failure, and re-run.

Symptom: Insider Risk, Communication Compliance, DSPM, Compliance Manager, or Unified Catalog show as **portal-only — review manually**. **Likely cause:** these have no reliable read cmdlet — this is expected, not an error. **Resolution:** open the exact portal location the tool records and capture current state by hand. Never substitute an invented value.

Symptom: a DLP policy's **mode** or a retention policy's **Enabled** state doesn't match the portal. **Likely cause:** stale session, or reading with a role that returns a partial view. **Resolution:** reconnect, re-read, and compare against the Purview portal DLP/retention list; the portal value is authoritative.

Remediation (simulation-first)

Symptom: `-WhatIf` prints nothing / the fix seems to do nothing. **Likely cause:** `-Commit` was omitted (correct behaviour — no change is intended without it), or `-PolicyName` doesn't match an existing policy. **Resolution:** confirm the policy name exactly (`Get-DlpCompliancePolicy | Select Name`), review the `-WhatIf` cmdlet line, then re-run with `-Commit`.

Symptom: enabling a retention policy is questioned by the records owner. **Likely cause:** whether a retention policy *should* run is a **records/compliance determination** (Requirement 2.6), not a technical fix. **Resolution:** get the records owner's written approval before `-Commit`; record it as evidence. If it was enabled prematurely, `PURVASSESS-Remediate.ps1 -Fix EnableRetentionPolicy -PolicyName "<policy>" -Revert -Commit`.

Symptom: a committed fix needs to be undone. **Resolution:** `PURVASSESS-Remediate.ps1 -Fix <name> -Revert -Commit` restores the recorded prior state from `PURVASSESS-Remediate-state.json`. Keep that state file with your evidence for the duration of the pilot.

Validation

Symptom: `PURVASSESS-Validate.ps1` reports FAIL after a committed fix. **Likely cause:** the change hasn't propagated yet (audit ingestion can take up to ~60 minutes), the wrong policy name was validated, or the commit didn't actually run. **Resolution:** wait for propagation and re-validate; confirm you passed the exact remediated policy names; if it truly didn't take, revert and investigate before expanding.

Scope of these gates

Every gate here proves the assessment **read the tenant correctly** and that a quick-win fix **took effect**. None of them judge whether a policy is *correct* or *sufficient for a regulation* — retention durations, DLP regulatory sufficiency, records applicability, and whether a capability is *required* are **compliance / records / legal determinations** routed to their owners (Requirement 2.6), not resolved here.

7. Best Practices — Assessing a Microsoft Purview Tenant

Operating principles for producing a Purview assessment that is trustworthy, actionable, and repeatable. Grounded in Microsoft's public guidance and the **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

Assess the whole estate, not one capability at a time

- The value of a *tenant* assessment is the cross-capability view — DLP without labels, labels without retention, Insider Risk without reviewers. Always run the full inventory, even for capabilities you assume are fine; the surprises are usually in the ones nobody looks at.
- Distinguish **not licensed**, **licensed-but-empty**, and **configured**. "Insider Risk is licensed but has zero policies" is a headline finding, not an omission — record it explicitly.

Read-only first; never let a permission gap look like a coverage gap

- Preflight roles and endpoints (`PURVASSESS-Scope.ps1`) **before** assessing. A capability that returns nothing because your account lacks the reader role is a false "gap" — the most common way an assessment misleads.
- Prefer **reader** roles for the assessment; grant **write** roles only for the specific quick-win capability you're remediating, under change control, and remove them after.

Be honest about what PowerShell can and can't read

- Several capabilities — **Insider Risk configuration, DSPM/DSPM for AI, Compliance Manager scoring, the Unified Catalog** — are **portal-driven** with limited or no stable PowerShell surface. Flag them **"portal-only — review manually"** with the exact portal location. **Never invent a cmdlet** to

manufacture a reading.

• Purview features are **licensing-dependent and the portal evolves**. Where a value can't be confirmed, write **"verify your licensing/tenant"** rather than asserting it. An honest "review manually" beats a confident wrong number.

Prioritize by risk, and make the priorities mean something

• Use a fixed, agreed scheme — **P1** (critical exposure / enforced-but-risky), **P2** (coverage gap / drift), **P3** (optimization) — and apply it consistently so the register is a plan, not a list.

• The point of the assessment is *which three things to fix first*. Resist the urge to present fifty equal findings; the readout should lead with the P1s.

Every fix is simulation/test/pilot-first — even the "safe" ones

• Moving a DLP policy's mode, enabling a retention policy, or taking auto-labeling to simulation are all reversible, but they are tenant-impacting. Apply them `-WhatIf` **first**, scoped to `Purview Assessment – Remediation Pilot – Stage1`, validate, then expand.

• **Never move a policy straight to enforce** as part of an assessment. The assessment's job is to *find and safely stabilize*, not to launch enforcement; full enforcement is a governed build step with its own pilot.

• Record the **prior state and the exact rollback** with every change so a questioned fix can be backed out cleanly (`-Revert -Commit`).

Keep the governance line bright

• Whether a retention duration is **correct**, whether a DLP rule **satisfies** a regulation, whether an Insider Risk policy **should exist** — these are **compliance/records/privacy/legal determinations**. Record them as findings **for the owner to decide**; the assessment supplies the evidence, not the verdict.

• **Insider Risk & Communication Compliance** are HR/privacy-sensitive — review **configuration hygiene and role separation only** (are reviewers assigned, is scope appropriate, is privacy-by-design/pseudonymization in place), never case content.

Make it a baseline you can re-run

• The first assessment is the baseline; the value compounds when you **re-run it on a cadence** (e.g. quarterly). Schedule the read-only chain and track the **P1/P2 trend** — a falling count is the objective measure that the estate is improving.

• Keep the register and scorecard as living artifacts owned by the compliance function, not a one-off PDF. Feed the non-remediated findings into implementation engagements as a governed backlog.

Least-privilege, evidenced, reversible

• Read with the minimum role; write only where and when a fix requires it.

• Evidence every finding with its source (which cmdlet/portal read produced it) so it's defensible and reproducible.

• Keep every change reversible and recorded — the assessment should leave the tenant **more stable and better understood**, never surprised.

8. Deliverables Summary — Microsoft Purview Tenant Assessment

What you receive, what to run, and what "done" looks like.

Documentation

Document	Purpose
1. Solution Overview	What a Purview tenant assessment is, the 11 capabilities it covers, how findings and priorities are produced, honest limitations, and the naming convention.
2. Requirements Guide	Assessment-specific prerequisites (licensing across capabilities, least-privilege **reader** roles, S&C/Exchange + Graph toolchain, confirmed capability inventory, write roles + pilot only if remediating) — each with Verify / Implement.
3. Implementation Guide	Ordered steps: preflight, inventory every enabled capability, design the findings/priority scheme, produce the register/scorecard, apply safe quick-win fixes simulation-first, validate, readout & track — pilot-first with pass/fail gates and rollback.
5. Testing & Validation	Ten gates from "preflight clean" to "readout delivered & backlog owned," each with method / pass / fail action.
6. Troubleshooting	Why a capability reads empty (permissions vs genuinely empty vs portal-only), licensing/latency effects, remediation and rollback issues — and their fixes.
7. Best Practices	Assess the whole estate, don't let a permission gap look like a coverage gap, be honest about portal-only capabilities, prioritize by risk, fix simulation-first, keep the governance line bright, re-run as a baseline.

PowerShell tools (5)

All discovery/validation/report tools are **read-only**; the remediation tool runs `-WhatIf` **by default** and writes only on `-Commit`, always simulation/test/pilot-first. No credentials or secrets are embedded. Labels/DLP/retention/eDiscovery reads use **Security & Compliance PowerShell** (`Connect-IPPSSession`); audit uses **Exchange Online PowerShell** (`Connect-ExchangeOnline`); licensing/roles use **Microsoft Graph**. Portal-only capabilities are flagged for manual review.

Script	Role	Default
`PURVASSESS-Scope.ps1`	Scope/health helper: licensing signals, per-capability read/role check, endpoint connectivity preflight.	Read-only
`PURVASSESS-Discovery.ps1`	Assess: inventory every enabled capability's configuration; tag each enabled / not enabled / not licensed / portal-only.	Read-only
`PURVASSESS-Remediate.ps1`	Configure/Remediate the safe quick wins (DLP→simulation, auto-label→simulation, enable disabled retention, enable unified audit) — simulation/test/pilot-first, with `-Revert`.	`-WhatIf` (writes on `-Commit`)
`PURVASSESS-Validate.ps1`	Validate: re-read the remediated capabilities and confirm each fix reached its target state.	Read-only
`PURVASSESS-Report.ps1`	Report: build the findings register, priority matrix (P1/P2/P3), and per-capability scorecard from the inventory.	Read-only

Named artifacts produced

- Findings register: `Purview Assessment – Tenant – Findings`.
- Priority matrix: `Purview Assessment – Priorities – P1P2P3`.
- Capability scorecard: `Purview Assessment – Capabilities – Scorecard`.
- Remediation pilot scope: `Purview Assessment – Remediation Pilot – Stage1`.
- Executive readout: `Purview Assessment – Tenant – Readout`.

Definition of done

- **Preflight clean** — licensing, reader roles, and endpoints confirmed; no permission gap masking a coverage gap.
- **Every enabled capability inventoried** — each of the 11 tagged enabled / not enabled / not licensed / portal-only, nothing silently skipped.
- **Findings register complete and prioritized** — each finding with capability, observation, evidence source, risk, recommendation, and P1/P2/P3.
- **Scorecard** reflects the inventory; **readout** delivered and walked with stakeholders.
- **Safe quick-win fixes** applied simulation/test/pilot-first, **validated green**, with rollback recorded.

• **Non-remediated findings** routed as a governed backlog (governance decision / build engagement / deferred) with owners, and the **re-baseline cadence scheduled**.

Boundary: *this solution delivers the technical assessment, prioritized findings, and safe stabilization.* ***Whether a policy is correct — retention durations, a rule's regulatory sufficiency, whether a capability is required, any records or legal-hold obligation — is deferred to your compliance, records, privacy, and legal functions.*** *Capability availability is* ***licensing-/tenant-dependent and evolving — verify against your entitlements and current Microsoft documentation.***

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PURVASSESS-Discovery.ps1

Purpose: Reads, with no changes, the current configuration of each Purview capability and tags it enabled / not enabled / not licensed / portal-only: Information Protection (labels, label policies, auto-labeling mode), DLP (policies, workloads, mode), Data Lifecycle & Records (retention policies/labels + enabled state), Audit (ingestion, retention), eDiscovery (cases, holds), Sensitive Info Types, and a manual-review pointer for the portal-only capabilities (Insider Risk, Communication Compliance, DSPM, Compliance Manager, Unified Catalog). Optionally writes the inventory to JSON for PURVASSESS-Report.ps1. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Discovery.ps1 -OutFile .\inventory.json Requires ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline). Interactive modern-auth / MFA admin sign-in; no secrets in code. EXPECTED RESULTS A per-capability inventory printed to the console and (with -OutFile) saved as JSON. Each capability shows a state and its key objects; unreadable/portal-only items are flagged, not asserted. .NOTES Purview category — Tenant Assessment. Read-only. Availability is licensing-/tenant-dependent and evolving; portal-only capabilities are flagged "review manually," never invented. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$OutFile # optional: path to save the inventory as JSON for the report tool)

How to run: powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Discovery.ps1 -OutFile .\inventory.json Requires ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline). Interactive modern-auth / MFA admin sign-in; no secrets in code. EXPECTED RESULTS A per-capability inventory printed to the console and (with -OutFile) saved as JSON. Each capability shows a state and its key objects; unreadable/portal-only items are flagged, not asserted. .NOTES Purview category — Tenant Assessment. Read-only. Availability is licensing-/tenant-dependent and evolving; portal-only capabilities are flagged "review manually," never invented. > [CmdletBinding()] param([string]$UserPrincipalName, [string]$OutFile # optional: path to save the inventory as JSON for the report tool)

PURVASSESS-Remediate.ps1

Purpose: Applies the low-risk, high-value remediations the assessment identifies — move a bluntly-enforced DLP policy to simulation, take an auto-labeling policy to simulation, enable a retention policy that was left disabled, or turn on unified audit ingestion. Every fix previews the exact cmdlet under -WhatIf, records the prior state so it is reversible, and changes state ONLY with -Commit. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Remediate.ps1 -Fix DlpToSimulation -PolicyName "Global PII" -WhatIf Review the -WhatIf output, then re-run with -Commit to apply. -Revert -Commit restores the recorded prior state. Requires ExchangeOnlineManagement (Connect-IPPSSession / Connect-ExchangeOnline) and the appropriate write role (Requirement 2.5). Interactive modern-auth / MFA; no secrets in code. EXPECTED RESULTS Under -WhatIf: prints the cmdlet it WOULD run and makes no change. Under -Commit: applies the fix, writes a prior-state record, and prints confirmation. A permissions error is surfaced, not swallowed. .NOTES Purview category — Tenant Assessment. Safe-by-default (-WhatIf). Pilot/simulation-first. Whether a policy SHOULD be enabled is a compliance/records determination (Requirement 2.6) — confirm first. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory = $true)]

[ValidateSet('DlpToSimulation','AutoLabelToSimulation','EnableRetentionPolicy','EnableUnifiedAudit')] [string]$Fix,

How to run: powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Remediate.ps1 -Fix DlpToSimulation -PolicyName "Global PII" -WhatIf Review the -WhatIf output, then re-run with -Commit to apply. -Revert -Commit restores the recorded prior state. Requires ExchangeOnlineManagement (Connect-IPPSSession / Connect-ExchangeOnline) and the appropriate write role (Requirement 2.5). Interactive modern-auth / MFA; no secrets in code. EXPECTED RESULTS Under -WhatIf: prints the cmdlet it WOULD run and makes no change. Under -Commit: applies the fix, writes a prior-state record, and prints confirmation. A permissions error is surfaced, not swallowed. .NOTES Purview category — Tenant Assessment. Safe-by-default (-WhatIf). Pilot/simulation-first. Whether a policy SHOULD be enabled is a compliance/records determination (Requirement 2.6) — confirm first. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory = $true)]
[ValidateSet('DlpToSimulation','AutoLabelToSimulation','EnableRetentionPolicy','EnableUnifiedAudit')] [string]$Fix,

PURVASSESS-Report.ps1

Purpose: Turns the PURVASSESS-Discovery.ps1 inventory into the assessment deliverables with no changes: a findings register (capability, observation, evidence source, risk, recommendation, priority), a P1/P2/P3 priority matrix, and a per-capability coverage/maturity scorecard. Findings that are regulatory/records/legal determinations are marked "decision: governance owner," not decided here. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Report.ps1 -InFile .\inventory.json -OutDir .\out Consumes the JSON from PURVASSESS-Discovery.ps1 -OutFile. Read-only; writes CSV/JSON deliverables to -OutDir only. EXPECTED RESULTS out\Purview-Assessment-Tenant-Findings.csv, out\Purview-Assessment-Priorities-P1P2P3.csv, out\Purview-Assessment-Capabilities-Scorecard.csv — matching the named artifacts in the docs. .NOTES Purview category — Tenant Assessment. Read-only. Priorities are heuristic starting points; final priority and any regulatory/records/legal determination are the customer's governance decision. > [CmdletBinding()] param([Parameter(Mandatory)][string]$InFile, # inventory JSON from PURVASSESS-Discovery.ps1 [string]$OutDir = '.' # where to write the deliverable CSVs)

How to run: powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Report.ps1 -InFile .\inventory.json -OutDir .\out Consumes the JSON from PURVASSESS-Discovery.ps1 -OutFile. Read-only; writes CSV/JSON deliverables to -OutDir only. EXPECTED RESULTS out\Purview-Assessment-Tenant-Findings.csv, out\Purview-Assessment-Priorities-P1P2P3.csv, out\Purview-Assessment-Capabilities-Scorecard.csv — matching the named artifacts in the docs. .NOTES Purview category — Tenant Assessment. Read-only. Priorities are heuristic starting points; final priority and any regulatory/records/legal determination are the customer's governance decision. > [CmdletBinding()] param([Parameter(Mandatory)][string]$InFile, # inventory JSON from PURVASSESS-Discovery.ps1 [string]$OutDir = '.' # where to write the deliverable CSVs)

PURVASSESS-Scope.ps1

Purpose: Preflights a Purview tenant assessment with no changes: reports which Purview-relevant service plans are present (E5 / E5 Compliance signals), which capability reads succeed vs throw a permissions error (so a role gap is not mistaken for a coverage gap), and which endpoints (Connect-IPPSSession, Connect-ExchangeOnline, Connect-MgGraph) are reachable. Read-only. HOW

TO RUN powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Scope.ps1 Requires the ExchangeOnlineManagement module (Connect-IPPSSession + Connect-ExchangeOnline) and Microsoft.Graph (Organization.Read.All / Directory.Read.All) for the licensing section. Interactive modern-auth / MFA admin sign-in; no secrets in code. EXPECTED RESULTS A preflight summary: reachable endpoints, licensed vs not for the key compliance plans, and a per-capability read OK / permission-denied / not-available list. Resolve any red before assessing. .NOTES Purview category — Tenant Assessment. Read-only. Capability availability is licensing-/tenant- dependent and evolving; items that cannot be confirmed are flagged "verify your licensing/tenant." > [CmdletBinding()] param([string]$UserPrincipalName, # optional admin UPN passed to Connect-* prompts [switch]$SkipGraph # skip the Graph licensing/roles section)

How to run: powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Scope.ps1 Requires the ExchangeOnlineManagement module (Connect-IPPSSession + Connect-ExchangeOnline) and Microsoft.Graph (Organization.Read.All / Directory.Read.All) for the licensing section. Interactive modern-auth / MFA admin sign-in; no secrets in code. EXPECTED RESULTS A preflight summary: reachable endpoints, licensed vs not for the key compliance plans, and a per-capability read OK / permission-denied / not-available list. Resolve any red before assessing. .NOTES Purview category — Tenant Assessment. Read-only. Capability availability is licensing-/tenant- dependent and evolving; items that cannot be confirmed are flagged "verify your licensing/tenant." > [CmdletBinding()] param([string]$UserPrincipalName, # optional admin UPN passed to Connect-* prompts [switch]$SkipGraph # skip the Graph licensing/roles section)

PURVASSESS-Validate.ps1

Purpose: Validation counterpart to PURVASSESS-Remediate.ps1. For each fix applied in Step 5 it re-reads the capability and confirms the target state: a DLP policy now in TestWithNotifications, an auto-labeling policy in simulation, a retention policy Enabled=True, and unified audit ingestion True. Read-only — it makes no changes; it only reports PASS/FAIL per fix so a finding can be marked cleared. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Validate.ps1 -DlpPolicy "Global PII" -RetentionPolicy "Legal Hold Baseline" -CheckAudit Pass the names of the policies you remediated. Requires ExchangeOnlineManagement (Connect-IPPSSession / Connect-ExchangeOnline). Interactive modern-auth / MFA; no secrets in code. EXPECTED RESULTS A PASS/FAIL line per checked fix. All PASS = the remediated findings validate green for the pilot scope (Gate 9). Any FAIL = the change did not take; re-check the fix / pilot scope before expanding. .NOTES Purview category — Tenant Assessment. Read-only. Confirms the fix took effect, NOT that the resulting configuration satisfies a regulation (that is a compliance/records/legal determination). > [CmdletBinding()] param([string]$DlpPolicy, # DLP policy expected in TestWithNotifications [string]$AutoLabelPolicy, # auto-labeling policy expected in simulation (TestWithoutNotifications) [string]$RetentionPolicy, # retention policy expected Enabled=True [switch]$CheckAudit, # verify UnifiedAuditLogIngestionEnabled=True [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\PURVASSESS-Validate.ps1 -DlpPolicy "Global PII" -RetentionPolicy "Legal Hold Baseline" -CheckAudit Pass the names of the policies you remediated. Requires ExchangeOnlineManagement (Connect-IPPSSession / Connect-ExchangeOnline). Interactive modern-auth / MFA; no secrets in code. EXPECTED RESULTS A PASS/FAIL line per checked fix. All PASS = the remediated findings validate green for the pilot scope (Gate 9). Any FAIL = the change did not take; re-check the fix / pilot scope before expanding. .NOTES Purview category — Tenant Assessment. Read-only. Confirms the fix took effect, NOT that the resulting configuration satisfies a regulation (that is a compliance/records/legal determination). > [CmdletBinding()] param([string]$DlpPolicy, # DLP policy expected in TestWithNotifications [string]$AutoLabelPolicy, # auto-labeling policy expected in simulation

(TestWithoutNotifications) [string]$RetentionPolicy, # retention policy expected Enabled=True [switch]$CheckAudit, # verify UnifiedAuditLogIngestionEnabled=True [string]$UserPrincipalName)